Департамент
образования и науки
города Москвы

Государственное бюджетное
профессиональное образовательное
учреждение города Москвы

Колледж информационных
технологий «ИТ.МОСКВА»

IT.МОСКВА

8 499 612-03-25
spo-mgkeit@edu.mos.ru

115446, г. Москва,
ул. Академика Миллионщикова, д. 20

00216071 ОКПО
1027739491790 ОГРН
7724035635 ИНН
772401001 КПП

ОТЧЁТ
по учебной практике

Специальность **09.02.07 Информационные системы и Программирование**
(код, наименование)

Вид практики Учебная практика УП.02

Сроки прохождения практики 29.09.26 - 12.10.26

Место прохождения практики колледж ИТ.МОСКВА
(название предприятия)

Выполнил студент третьего курса группы **3-ИП-7-24**

Тур Глеб Константинович
(Ф.И.О. студента)

Москва, 2026

Занятие 1. Определение границ операционной системы и разработка нефункциональных требований

1.1. Цель занятия

Определить границы учебной операционной системы my_os, разработать сценарии использования и сформулировать нефункциональные требования с измеримыми метриками.

1.2. Название и концепция операционной системы

Название проекта: my_os - учебная операционная система с монолитным ядром.

Концепция: учебная ОС реализует базовые подсистемы управления ресурсами вычислительной системы: процессы, память, файловую систему, разграничение прав пользователей и командную оболочку. В качестве физического носителя данных используется встроенная СУБД SQLite. Реальная работа с аппаратным обеспечением заменена программными заглушками. Основной акцент сделан на подсистеме планирования процессов: реализуются алгоритмы FCFS, Round Robin и приоритетного планирования, а также сравнение их эффективности по среднему времени ожидания и времени отклика.

1.3. Границы системы

1.3.1. Что входит в границы ОС

№	Компонент	Описание
1	Управление процессами	Создание, запуск, завершение процессов; ведение таблицы процессов; базовое планирование
2	Управление памятью	Выделение и освобождение памяти процессам; учёт лимитов; имитация виртуальной памяти
3	Файловая система на SQLite	Создание, чтение, запись, удаление файлов и каталогов; хранение в таблице files
4	Разграничение прав пользователей	Аутентификация по логину/паролю (SHA-256), роли: обычный пользователь, администратор

5	Командная оболочка (shell)	Цикл чтения команд, вызов системных вызовов, вывод результата
6	Подсистема журналирования	Запись всех системных вызовов в таблицу syscalls_log
7	Планировщик процессов	Реализация алгоритмов FCFS, Round Robin, приоритетного планирования; учёт времени ожидания

1.3.2. Что не входит в границы ОС

№	Исключённый компонент	Обоснование
1	Реальная работа с аппаратным обеспечением	Требует глубокого знания архитектуры; в учебной ОС заменяется заглушками
2	Поддержка прерываний и драйверов	Сложность реализации и отладки; выходит за рамки отведённого времени
3	Сетевые протоколы	Не является темой проекта; требует отдельной подсистемы
4	Графический интерфейс	Достаточно командной оболочки; GUI - отдельная большая тема
5	Интеграция с LDAP/AD	Избыточно для учебного проекта; аутентификация реализуется локально

1.4. Сценарии использования

Роли пользователей: обычный пользователь, администратор, система (фоновые процессы).

№	Сценарий	Действующее лицо	Системные вызовы	Модули

1	Вход в систему	Пользователь	sys_login	shell.py, syscalls.py, db.py (users)
2	Создание файла	Пользователь	sys_create	shell.py, syscalls.py, db.py (files)
3	Чтение файла	Пользователь	sys_open → sys_read	shell.py, syscalls.py, db.py
4	Запуск процесса	Пользователь	sys_exec	kernel.py, syscalls.py, db.py (processes)
5	Просмотр списка процессов	Администратор	sys_ps	syscalls.py, db.py
6	Завершение процесса	Администратор	sys_kill	kernel.py, syscalls.py, db.py
7	Просмотр журнала системных вызовов	Администратор	sys_logs	syscalls.py, db.py
8	Выделение памяти процессу	Система	sys_alloc	kernel.py, syscalls.py, db.py
9	Завершение работы ОС	Администратор	sys_shutdown	shell.py, syscalls.py, kernel.py
10	Постановка процесса в очередь планировщика	Система	sys_enqueue	kernel.py, syscalls.py, db.py
11	Смена приоритета процесса	Администратор	sys_setprio	kernel.py, syscalls.py, db.py

1.5. Нефункциональные требования

1.5.1. Производительность

№	Требование	Метрика	Способ проверки
1	Максимальное время обработки одного системного вызова	≤ 50 мс	time.perf_counter()
2	Количество одновременно поддерживаемых процессов	≥ 10	Стресс-тест: запуск 10 процессов
3	Время отклика командной оболочки на команду help	≤ 100 мс	Ручное измерение
4	Среднее время ожидания процесса в очереди планировщика	≤ 200 мс	Замер по 100 запускам
5	Время переключения контекста между процессами	≤ 10 мс	time.perf_counter() вокруг планировщика

1.5.2. Надёжность

№	Требование	Метрика
1	Ядро не должно аварийно завершаться при ошибке в пользовательском процессе	100% перехват исключений
2	При ошибке процесс останавливается, запись в журнал	Наличие записи в syscalls_log
3	Реакция на попытку выделения памяти сверх лимита	Отказ + запись в журнал

1.5.3. Безопасность

№	Требование	Метрика
1	Пароли хранятся только в виде SHA-256	Отсутствие открытых паролей в БД
2	Обычный пользователь не может изменять/удалять файлы администратора	Проверка прав доступа
3	Все системные вызовы фиксируются в журнале	Запись: вызов, аргументы, пользователь, результат

1.5.4. Поддерживаемость

№	Требование	Метрика
1	Модульность кода	Каждый модуль $\leq$ 300 строк
2	Разделение ответственности	Отдельные файлы: kernel.py, syscalls.py, shell.py, db.py, config.py

1.5.5. Ресурсоёмкость

№	Требование	Метрика
1	Работа на стандартном Python 3.10+	Без дополнительных серверов БД
2	Использование только встроенных библиотек	sqlite3, hashlib, os, sys

1.6. Анализ рисков

№	Риск	Стратегия минимизации

1	Нехватка времени	Приоритизация: ядро и БД → файловая система → оболочка → дополнительные функции
2	Блокировки SQLite при одновременных обращениях	Единственное подключение + очередь запросов
3	Сложность отладки системных вызовов	Сквозное логирование всех вызовов в syscalls_log
4	Забывчивость в обработке ошибок	Чек-лист: проверка try/except в каждом модуле
5	Потеря целостного представления о проекте	Ведение подробной документации и регулярная сверка с ней
6	Ошибки в хэшировании паролей	Использование проверенного hashlib.sha256, юнит-тесты
7	Некорректная работа планировщика	Модульные тесты, логирование состояний
8	Неэффективный алгоритм планирования	Сравнение FCFS, Round Robin и приоритетного на тестовых данных
9	Голодание низкоприоритетных процессов	Введение старения (aging) или квоты времени

1.7. Описание созданного прототипа

№	Модуль	Назначение	Как проверить
1	src/	Подключение к SQLite, инициализация таблиц	python src/db.py →

	db.py	users, processes, files, syscalls_log	файл db/os.sqlite
2	src/ syscalls.py	Реализация sys_echo, sys_get_users, логирование	Вызов из оболочки, проверка syscalls_log
3	src/ shell.py	Командная оболочка: help, echo, users, exit	python -m src.shell
4	src/ kernel.py	Ядро: управление процессами, памятью (заготовка)	Импорт и вызов функций
5	src/ config.py	Конфигурация: путь к БД, лимиты	Импорт констант

1.8. Словарь терминов

№	Термин	Определение
1	Ядро (kernel)	Совокупность модулей, постоянно находящихся в ОЗУ и управляющих ресурсами системы
2	Пространство ядра	Защищённая область памяти, доступная только модулям ядра
3	Пространство пользователя	Область, в которой работают прикладные программы и оболочка
4	Системный вызов	Формализованный интерфейс запроса привилегированной операции у ядра
5	Процесс	Программа в момент выполнения, обладающая собственным состоянием

6	Поток	Легковесная единица выполнения внутри процесса
7	Планировщик	Модуль, определяющий, какой процесс будет выполняться в данный момент
8	Виртуальная память	Механизм, при котором адреса процесса не совпадают с физическими
9	Дескриптор файла	Числовой идентификатор открытого файла
10	Хэширование	Необратимое преобразование данных в строку фиксированной длины
11	Монолитное ядро	Архитектура, при которой все модули работают в едином адресном пространстве
12	Аутентификация	Проверка подлинности пользователя
13	Авторизация	Определение, какие действия разрешены пользователю
14	Прерывание	Сигнал от аппаратного устройства или ПО, требующий внимания ядра
15	Драйвер	Программный модуль взаимодействия ядра с периферией

[ВСТАВЬ СЮДА: скриншот структуры проекта my_os в VS Code]

1.9. Вывод по занятию 1

В ходе занятия 1 были определены границы учебной ОС my_os, разработаны 11 сценариев использования, сформулированы нефункциональные требования по пяти категориям с измеримыми метриками, проведён анализ рисков (9 рисков), создан базовый прототип (db.py, syscalls.py, shell.py), оформлен словарь

терминов из 15 понятий.

Занятие 2. Архитектурное проектирование и проектирование API ядра

2.1. Цель занятия

Спроектировать модульную архитектуру ядра учебной ОС, разработать API системных вызовов, реализовать заглушки и логирование.

2.2. Название и концепция ОС

Название: StudyOS - учебная операционная система с монолитным ядром.

Концепция аналогична занятию 1, но с уточнённой архитектурой и расширенным набором системных вызовов (13 заглушек).

2.3. Архитектура системы

Модульная архитектура StudyOS включает следующие модули:

- `shell.py` - командная оболочка;
- `syscalls.py` - реализация системных вызовов и логирования;
- `kernel.py` - ядро: обработка вызовов, проверка прав;
- `scheduler.py` - планировщик процессов (FCFS, Round Robin, приоритетный);
- `memory.py` - управление памятью;
- `fs.py` - файловая система на SQLite;
- `auth.py` - аутентификация и авторизация;
- `db.py` - подключение к SQLite и инициализация таблиц;
- `config.py` - конфигурация.

Направления вызовов:

- shell.py → syscalls.py
- syscalls.py → kernel.py
- kernel.py → scheduler.py, memory.py, fs.py, auth.py
- scheduler.py, memory.py, fs.py, auth.py → db.py

Важно: оболочка shell.py не обращается к db.py напрямую. Если БД изменится, оболочка об этом не узнает - это правильная архитектура.

[ВСТАВЬ СЮДА: диаграмма компонентов StudyOS]

2.4. Реализация системных вызовов

Файл `src/syscalls.py` содержит функцию логирования и 13 заглушек системных вызовов. Каждая функция вызывает `log_syscall()` и возвращает правдоподобный результат.

2.4.1. Функция логирования

```
def log_syscall(name, args="", user="system", status="OK"):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO syscalls_log (syscall_name, args, user, status) "
        "VALUES (?, ?, ?, ?)",
        (name, args, user, status)
    )
    conn.commit()
    conn.close()
```

2.4.2. Заглушки системных вызовов

```
def sys_login(login, password, current_user="guest"):
    log_syscall("sys_login", login, current_user)
    return True

def sys_logout(current_user="guest"):
    log_syscall("sys_logout", "", current_user)
    return True

def sys_whoami(current_user="guest"):
    log_syscall("sys_whoami", "", current_user)
    return current_user

def sys_create_file(path, content, current_user="guest"):
    log_syscall("sys_create_file", path, current_user)
    return 1

def sys_read_file(path, current_user="guest"):
    log_syscall("sys_read_file", path, current_user)
    return ""

def sys_delete_file(path, current_user="guest"):
    log_syscall("sys_delete_file", path, current_user)
    return True

def sys_list_files(path, current_user="guest"):
    log_syscall("sys_list_files", path, current_user)
    return []

def sys_exec(program, current_user="guest"):
    log_syscall("sys_exec", program, current_user)
    return 4
```

```
def sys_ps(current_user="guest"):
    log_syscall("sys_ps", "", current_user)
    return []

def sys_kill(pid, current_user="guest"):
    log_syscall("sys_kill", str(pid), current_user)
    return True

def sys_mem_alloc(size, current_user="guest"):
    log_syscall("sys_mem_alloc", str(size), current_user)
    return 1

def sys_logs(current_user="guest"):
    log_syscall("sys_logs", "", current_user)
    return []

def sys_shutdown(current_user="guest"):
    log_syscall("sys_shutdown", "", current_user)
    return True
```

2.4.3. Проверка логирования

После запуска `python -m src.syscalls` в таблице `syscalls_log` появляются 4 новые записи.

№	syscall_name	args	user	status
1	sys_login	admin	admin	OK
2	sys_whoami		admin	OK
3	sys_create_file	/test.txt	admin	OK
4	sys_ps		admin	OK

2.5. Обновление оболочки

Файл `src/shell.py` расширен новыми командами: `whoami`, `login`, `logout`, `create`, `ls`, `ps`, `logs`.

Пример сеанса работы:

```
guest@studyos:~$ whoami
guest
guest@studyos:~$ login
Логин: admin
Пароль: secret
Вы вошли как admin
admin@studyos:~$ create
Путь: /test.txt
Содержимое: hello
Создан файл с id=1
admin@studyos:~$ ls
admin@studyos:~$ ps
admin@studyos:~$ exit
```

2.6. Git-коммиты

Изменения за день фиксируются минимум двумя коммитами: промежуточным после проектирования и финальным после реализации.

2.7. Итоговые артефакты

№	Артефакт	Где лежит	Что содержит
1	Проектные артефакты	docs/ 02_OS_Architecture.md	Список модулей, спецификация вызовов, контракты, структуры данных
2	Программные артефакты	src/syscalls.py, src/shell.py	log_syscall, 13 заглушек, команды whoami, login, create, ls, ps
3	Артефакт БД	db/os.sqlite	Таблица syscalls_log минимум с 4 записями
4	Артефакт Git	Репозиторий my_os	Минимум два коммита за день

2.8. Словарь терминов

№	Термин	Определение

1	Ядро (kernel)	Совокупность модулей, управляющих ресурсами системы
2	Пространство ядра	Защищённая область памяти, доступная только ядру
3	Пространство пользователя	Область для прикладных программ и оболочки
4	Системный вызов (syscall)	Формализованный интерфейс запроса привилегированной операции
5	Процесс	Программа в момент выполнения
6	Поток (thread)	Легковесная единица выполнения внутри процесса
7	Планировщик (scheduler)	Модуль, определяющий, какой процесс будет выполняться
8	Виртуальная память	Механизм, при котором адреса процесса не совпадают с физическими
9	Дескриптор файла	Числовой идентификатор открытого файла
10	Хэширование	Необратимое преобразование данных в строку фиксированной длины
11	Монолитное ядро	Архитектура, при которой все модули работают в едином адресном пространстве
12	Аутентификация	Проверка подлинности пользователя

13	Авторизация	Определение прав пользователя
14	Прерывание	Сигнал от аппаратного устройства или ПО
15	Драйвер	Модуль взаимодействия ядра с периферией

2.9. Вывод по занятию 2

В ходе занятия 2 спроектирована модульная архитектура ядра StudyOS, разработан API из 13 системных вызовов, реализованы заглушки с логированием, обновлена командная оболочка, создана таблица syscalls_log с записями о вызовах. Архитектура обеспечивает правильное разделение ответственности: оболочка не работает с БД напрямую.

Практическая работа №3. Построение архитектуры программного средства

3.1. Постановка задачи

Предметная область: кадровое агентство.

Кадровое агентство способствует трудоустройству безработных граждан. Агентство ведёт учёт и классификацию данных о безработных на основании резюме от них. От предприятий города поступают данные о свободных вакансиях, на основании которых агентство предлагает различные варианты трудоустройства соискателям. В случае положительного исхода поиска вакансия считается заполненной, а безработный становится трудоустроенным. По результатам своей деятельности кадровое агентство производит отчисления в налоговые органы и предоставляет отчётность в органы государственной статистики.

Цель разработки: автоматизация учёта безработных граждан и вакансий, подбор вариантов трудоустройства, оформление направлений, фиксация трудоустройства, формирование отчётности.

3.2. Анализ функциональных требований

1. Ведение базы резюме безработных.
2. Ведение базы вакансий от предприятий.
3. Классификация резюме и вакансий по категориям.
4. Автоматический подбор вакансий под резюме.

5. Оформление направления на вакансию.
6. Фиксация трудоустройства.
7. Формирование отчётности для налоговых органов и госстатистики.
8. Разграничение прав доступа (администратор, инспектор, бухгалтер).
9. Поиск и фильтрация данных по заданным критериям.
10. Хранение истории действий и изменений.

3.3. Анализ эксплуатационных требований

- **Надёжность:** устойчивая работа, контроль входных и выходных данных, восстановление после отказа не более 1 часа, ежедневное резервное копирование БД.
- **Условия эксплуатации:** офисное помещение, температура 18-25 °C, относительная влажность 40-70 %.
- **Технические средства:** сервер (4 ядра, 16 ГБ ОЗУ, SSD 500 ГБ), рабочие станции (2 ядра, 8 ГБ ОЗУ, SSD 256 ГБ), МФУ, сетевое оборудование.
- **Программная совместимость:** Windows 10/11, Linux Ubuntu Server 22.04, PostgreSQL 15, Python 3.11 (Django/Flask), Nginx/Apache.
- **Персонал:** администратор, инспекторы, бухгалтер.

3.4. Эскизный проект

3.4.1. Выбор метода решения и языка программирования

- **Метод решения:** структурный подход, нисходящее проектирование, метод пошаговой детализации.
- **Язык программирования:** Python 3.11.
- **Фреймворк:** Django (веб-приложение).
- **СУБД:** PostgreSQL 15.
- **Средства проектирования:** MS Visio, draw.io, MS Office.

Обоснование: Python обеспечивает быструю разработку и кроссплатформенность; Django даёт готовую админ-панель и ORM; PostgreSQL - надёжная реляционная СУБД с поддержкой транзакций.

3.4.2. Структура программного продукта

1. Подсистема авторизации и разграничения доступа.
2. Подсистема учёта безработных (резюме).
3. Подсистема учёта вакансий.
4. Подсистема подбора и трудоустройства.
5. Подсистема отчётности.
6. Подсистема администрирования.

3.4.3. Состав функций программного продукта

- F1 - добавление/редактирование/удаление резюме.
- F2 - добавление/редактирование/удаление вакансии.
- F3 - поиск и фильтрация резюме и вакансий.

- F4 - автоматический подбор вакансий под резюме.
- F5 - оформление направления на вакансию.
- F6 - фиксация трудоустройства.
- F7 - формирование отчётов.
- F8 - управление пользователями и правами доступа.

3.4.4. Режимы функционирования

- **Основной режим** - работа с БД в реальном времени.
- **Режим отчётности** - формирование и выгрузка отчётов (PDF, XLSX).
- **Режим администрирования** - управление пользователями, резервное копирование.
- **Режим обслуживания** - обновление, миграция данных, восстановление БД.

3.5. Спецификации процессов

3.5.1. Процесс «Добавление резюме»

Вход: данные соискателя (ФИО, контакты, образование, опыт, желаемая должность, зарплата).
Выход: запись в таблице «Резюме», уведомление об успехе.

Алгоритм:

1. Проверить корректность введённых данных.
2. Проверить наличие дубликата в БД.
3. Если дубликат есть - вывести сообщение об ошибке.
4. Иначе - добавить запись в БД.
5. Вернуть результат операции.

3.5.2. Процесс «Подбор вакансии»

Вход: ID резюме, критерии подбора.
Выход: список подходящих вакансий.

Алгоритм:

1. Получить данные резюме по ID.
2. Выполнить SQL-запрос к таблице «Вакансии» по критериям.
3. Отсортировать результат по степени соответствия.
4. Вернуть список вакансий.

3.5.3. Процесс «Оформление трудоустройства»

Вход: ID резюме, ID вакансии.
Выход: запись в таблице «Трудоустройство», статус вакансии «заполнена».

Алгоритм:

1. Проверить, свободна ли вакансия.
2. Проверить, не трудоустроен ли уже соискатель.
3. Создать запись о трудоустройстве.

4. Изменить статус вакансии на «заполнена».
5. Изменить статус соискателя на «трудоустроен».
6. Вернуть результат.

3.5.4. Процесс «Формирование отчёта»

Вход: тип отчёта, период.
Выход: файл отчёта в PDF/XLSX.

Алгоритм:

1. Принять параметры отчёта.
2. Выполнить выборку данных из БД.
3. Сформировать отчёт по шаблону.
4. Сохранить в файл и вернуть пользователю.

3.6. Диаграммы потоков данных (DFD)

3.6.1. Контекстная диаграмма (уровень 0)

Внешние сущности: соискатель, предприятие, налоговые органы, органы госстатистики. Основной процесс - «ИС Кадровое агентство».

[ВСТАВЬ СЮДА: DFD уровень 0]

3.6.2. Диаграмма уровня 1

Процессы: P1 - Учёт безработных, P2 - Учёт вакансий, P3 - Подбор и трудоустройство, P4 - Формирование отчётности. Хранилища: D1 - Резюме, D2 - Вакансии, D3 - Трудоустройство, D4 - Пользователи.

[ВСТАВЬ СЮДА: DFD уровень 1]

3.6.3. Диаграмма уровня 2

Декомпозиция P3: P3.1 - Поиск подходящих вакансий, P3.2 - Оформление направления, P3.3 - Фиксация трудоустройства.

[ВСТАВЬ СЮДА: DFD уровень 2]

3.7. Диаграмма «сущность-связь» (ERD)

Сущности: Соискатель, Резюме, Предприятие, Вакансия, Направление, Трудоустройство, Пользователь, Отчёт.

Связи: Соискатель (1) - (M) Резюме; Предприятие (1) - (M) Вакансия; Резюме (M) - (M) Вакансия (через Направление); Соискатель (1) - (M) Трудоустройство; Вакансия (1) - (M) Трудоустройство; Пользователь (1) - (M) Отчёт.

[ВСТАВЬ СЮДА: ERD]

3.8. Словарь терминов

Термин	Определение
Соискатель	Безработный гражданин, ищущий работу через агентство
Резюме	Документ с данными о соискателе
Вакансия	Свободное рабочее место от предприятия
Предприятие	Организация, предоставившая вакансию
Направление	Документ, направляющий соискателя на собеседование
Трудоустройство	Факт приёма соискателя на вакансию
DFD	Data Flow Diagram - диаграмма потоков данных
ERD	Entity-Relationship Diagram - диаграмма «сущность-связь»
СУБД	Система управления базами данных

3.9. Технический проект

3.9.1. Структурная схема

[ВСТАВЬ СЮДА: структурная схема]

3.9.2. Функциональная схема

[ВСТАВЬ СЮДА: функциональная схема]

3.9.3. Алгоритм программы

[ВСТАВЬ СЮДА: блок-схема алгоритма]

3.9.4. Структурная карта Константайна

[ВСТАВЬ СЮДА: карта Константайна]

3.9.5. Структурная карта Джексона

[ВСТАВЬ СЮДА: карта Джексона]

3.9.6. Законченный технический проект программного модуля

Модуль «Подбор и трудоустройство».

Назначение: автоматический подбор вакансий под резюме, оформление направления и фиксация трудоустройства.

Входные данные: ID резюме, критерии подбора, ID вакансии.

Выходные данные: список вакансий, запись в «Направление», запись в «Трудоустройство», обновлённые статусы.

Требования: время отклика $\leq$ 2 с, поддержка до 1000 одновременных запросов, логирование, разграничение прав.

3.10. Вывод по ПР№3

Построена архитектура программного средства для ИС «Кадровое агентство»: выполнен анализ функциональных и эксплуатационных требований, разработан эскизный проект, построены DFD уровней 0-2, ERD, структурная и функциональная схемы, алгоритм, карты Константайна и Джексона, а также законченный технический проект модуля.

Практическая работа №4. Изучение работы в системе контроля версий

4.1. Цель работы

Применение методов объектно-ориентированного проектирования и изучение работы в системе контроля версий Git.

4.2. Постановка задачи

Предметная область: автосалон «START», продажа автомобилей.

Требуется разработать программу на языке Visual Basic, реализующую объектно-ориентированный подход:

- выбор фирмы-изготовителя;
- определение марки автомобиля;
- определение стоимости в долларах США;
- просмотр изображения и характеристик;
- получение полной информации.

Дополнительно: освоить Git (init, commit, branch, merge, log).

4.3. Теоретические сведения об ООП

Сущность объектно-ориентированного подхода:

- программа - модель предметной области;
- модель - совокупность взаимодействующих объектов;
- объект описывается параметрами и операциями;
- взаимодействие - через сообщения;
- объекты с одинаковым набором параметров и операций - класс.

Классы могут образовывать иерархическую структуру. Производный класс наследует свойства базового. Некоторые параметры объекта могут быть локализованы внутри и недоступны извне (инкапсуляция).

4.4. Разработка класса Class1

4.4.1. Поля класса

Элемент	Тип	Назначение
avto	Boolean	Флаг согласия на покупку
firma	String	Фирма-изготовитель
model	String	Марка автомобиля
stoim	String	Стоимость в долларах США
pict	String	Номер элемента массива PictureBox
var	String	Режим вывода

4.4.2. Методы класса

- `Met1()` - проверяет согласие на покупку, вызывает Met2, Met3, Met4;
- `Met2()` - определяет марку автомобиля по фирме;
- `Met3()` - определяет стоимость автомобиля;
- `Met4()` - определяет номер элемента массива PictureBox;
- `Property Get varian()` - чтение свойства varian;
- `Property Let varian()` - запись свойства varian.

4.4.3. Код класса Class1

```
Public avto As Boolean
Public firma As String
Public model As String
Public stoim As String
Public pict As String
Dim var As String

Private Sub Class_Initialize()
    avto = False: firma = "": model = "": stoim = "": var = ""
End Sub

Public Function Met1()
    If avto = True Then
        model = Met2()
        stoim = Met3()
        pict = Met4()
        Met1 = True
    Else
        Met1 = False
    End If
End Function

Public Function Met2()
    Select Case firma
        Case "AUDI": Met2 = "A6"
        Case "CITROEN": Met2 = "C5"
        Case "FORD": Met2 = "Focus"
        Case "HONDA": Met2 = "Accord"
        Case "HYUNDAI": Met2 = "Elanta"
        Case "JEEP": Met2 = "Grand Cherokee LTD"
        Case "LAND ROVER": Met2 = "Land Rover Discovery"
        Case "LEXSUS": Met2 = "RX330"
        Case "MITSUBISHI": Met2 = "Pajero III"
        Case "NISSAN": Met2 = "Primera(1.8)"
        Case "PEUGEOT": Met2 = "307 XR"
        Case "PORSCHE": Met2 = "Cayenne Turbo"
        Case "RENAULT": Met2 = "Laguna II"
    End Select
End Function

Public Function Met3()
    Select Case firma
        Case "AUDI": Met3 = "41500"
        Case "CITROEN": Met3 = "20100"
```

```
        Case "FORD": Met3 = "12430"
        Case "HONDA": Met3 = "33900"
        Case "HYUNDAI": Met3 = "13790"
        Case "JEEP": Met3 = "41690"
        Case "LAND ROVER": Met3 = "40850"
        Case "LEXSUS": Met3 = "65500"
        Case "MITSUBISHI": Met3 = "56640"
        Case "NISSAN": Met3 = "25100"
        Case "PEUGEOT": Met3 = "13808"
        Case "PORSCHE": Met3 = "140500"
        Case "RENAULT": Met3 = "22900"
    End Select
End Function

Public Function Met4()
    Select Case firma
        Case "AUDI": Met4 = "0"
        Case "CITROEN": Met4 = "1"
        Case "FORD": Met4 = "2"
        Case "HONDA": Met4 = "3"
        Case "HYUNDAI": Met4 = "4"
        Case "JEEP": Met4 = "5"
        Case "LAND ROVER": Met4 = "6"
        Case "LEXSUS": Met4 = "7"
        Case "MITSUBISHI": Met4 = "8"
        Case "NISSAN": Met4 = "9"
        Case "PEUGEOT": Met4 = "10"
        Case "PORSCHE": Met4 = "11"
        Case "RENAULT": Met4 = "12"
    End Select
End Function

Public Property Get varian() As String
    Select Case var
        Case Is = 0: varian = "pict"
        Case Is = 1: varian = "texn"
        Case Is = 2: varian = "all"
    End Select
End Property

Public Property Let varian(ByVal vNewValue As String)
    Select Case vNewValue
        Case "изображение": var = 0
        Case "технические параметры": var = 1
        Case Else: var = 2
```

```
    End Select
End Property
```

4.5. Разработка формы Form1

4.5.1. Элементы управления

Элемент	Имя	Назначение
ComboBox	Combo1	Выбор фирмы-изготовителя
ComboBox	Combo2	Выбор режима вывода
CommandButton	Command1	Кнопка «OK»
CommandButton	Command2	Кнопка «Exit»
Label	Label1 - Label5	Текстовые надписи
OptionButton	Option1(0), Option1(1)	Переключатели «да»/«нет»
PictureBox	Picture1(0..12)	Изображения автомобилей
PictureBox	Picture2(0..12)	Технические характеристики

4.5.2. Свойства элементов

- `Command1.Caption = "OK"`, `Command2.Caption = "Exit"`;
- `Label1.Caption = "Хотите ли Вы купить машину?"`;
- `Label2.Caption = "Выберите фирму-изготовитель"`;
- `Label3.Caption = "Хотите ли Вы увидеть изображение..."`;
- `Label4.Caption = ""`, `Label4.Visible = False`;
- `Option1(0).Caption = "да"`, `Option1(1).Caption = "нет"`;
- Visible всех PictureBox = False.

[ВСТАВЬ СЮДА: скриншот формы Form1]

4.5.3. Код формы Form1

```
Dim av As Class1
Dim v As String
Dim i As Integer, j As Integer

Private Sub Combo1_Click()
    Label5.Visible = False
    For i = 0 To 12
        Picture1(i).Visible = False
        Picture2(i).Visible = False
    Next
    Dim ot As String
    av.firma = Combo1.Text
    av.avto = Option1(0).Value
    If av.Met1 = True Then
        ot = " " & CStr(av.firma) & vbCrLf: ot = ot & " " & vbCrLf
        ot = ot & " модель " & CStr(av.model) & vbCrLf
        ot = ot & " цена в $ " & CStr(av.stoim) & vbCrLf
        ot = ot & "Для получения более полной информации обращайтесь по телефону
7077888"
        MsgBox Prompt:=ot, Title:="Мы можем предложить"
    Else
        Label5.Visible = False
        Picture1(Val(av.pict)).Visible = False
         ot = "Приносим свои извинения, мы даем информацию для желающих купить
автомобиль"
        MsgBox Prompt:=ot, Title:="Автосалон START"
    End If
End Sub

Private Sub Combo2_Click()
    av.varian = Combo2.Text
End Sub

Private Sub Command1_Click()
    Label5.Visible = False
    Label5.Caption = ""
    For i = 0 To 12
        Picture1(i).Visible = False
        Picture2(i).Visible = False
    Next
    v = av.varian
    av.avto = Option1(0).Value
    If av.Met1 = True Then
        Select Case v
            Case "pict"
```

```
                Picture1(Val(av.pict)).Visible = True
            Case "texn"
                Picture2(Val(av.pict)).Visible = True
            Case "all"
                Picture1(Val(av.pict)).Visible = True
                Picture2(Val(av.pict)).Visible = True
                Label5.Visible = True
               Label5.Caption = CStr(av.firma) & " " & CStr(av.model) & vbCrLf &
"цена в $ " & CStr(av.stoim)
        End Select
    Else
        Picture1(Val(av.pict)).Visible = False
        Picture2(Val(av.pict)).Visible = False
        MsgBox Prompt:="Приносим свои извинения...", Title:="Автосалон START"
    End IfEnd Sub

Private Sub Command2_Click()
       MsgBox Prompt:="Мы всегда рады помочь! Будем рады новой встрече!",
Title:="Автосалон START"
    End
End Sub

Private Sub Form_Load()
    Set av = New Class1
    Combo1.AddItem "AUDI"
    Combo1.AddItem "CITROEN"
    Combo1.AddItem "FORD"
    Combo1.AddItem "HONDA"
    Combo1.AddItem "HYUNDAI"
    Combo1.AddItem "JEEP"
    Combo1.AddItem "LAND ROVER"
    Combo1.AddItem "LEXSUS"
    Combo1.AddItem "MITSUBISHI"
    Combo1.AddItem "NISSAN"
    Combo1.AddItem "PEUGEOT"
    Combo1.AddItem "PORSCHE"
    Combo1.AddItem "RENAULT"
    Combo2.AddItem "изображение"
    Combo2.AddItem "технические параметры"
    Combo2.AddItem "все"
End Sub
```

4.6. Работа в системе контроля версий Git

Порядок работы:

1. Установить Git.
2. Создать папку проекта.
3. Инициализировать репозиторий: `git init`.
4. Настроить пользователя: `git config --global user.name "Student"`, `git config --global user.email "..."`.
5. Добавить файлы: `git add .`.
6. Первый коммит: `git commit -m "Initial commit: класс Class1 и форма Form1"`.
7. Создать ветку: `git branch feature-add-toyota`, `git checkout feature-add-toyota`.
8. Внести изменения (добавить фирму TOYOTA), закоммитить.
9. Вернуться в main и слить: `git checkout main`, `git merge feature-add-toyota`.
10. Просмотреть историю: `git log --oneline --graph --all`.

[ВСТАВЬ СЮДА: скриншот git log --oneline --graph --all]

[ВСТАВЬ СЮДА: скриншот git branch]

[ВСТАВЬ СЮДА: скриншот git status]

4.7. Результаты работы

В ходе выполнения работы:

- разработан класс Class1 с полями avto, firma, model, stoim, pict, var;
- реализованы методы Met1, Met2, Met3, Met4;
- реализованы свойства varian (Property Get и Property Let);
- разработана форма Form1 с элементами управления;
- написаны обработчики событий Combo1_Click, Combo2_Click, Command1_Click, Command2_Click, Form_Load;
- создан Git-репозиторий, выполнены коммиты, создана и слита ветка.

4.8. Вывод по ПР№4

Изучен объектно-ориентированный подход на примере разработки программы «Автосалон START» на языке Visual Basic. Разработан класс Class1 с полями, методами и свойствами, реализована форма Form1. Дополнительно изучена работа в Git: создание репозитория, коммиты, ветвление, слияние, просмотр истории.

Практическая работа №5. Основы Python в VS Code. Структуры данных для предметной области «Чудо Обувь»

5.1. Цель занятия

Научиться работать в VS Code, освоить базовые конструкции Python и структуры данных для реализации приложения «Чудо Обувь».

5.2. Теоретическая часть

5.2.1. Переменные и типы данных

```
name = "Air Max"       # строка
price = 8990.0         # дробное число
quantity = 5           # целое число
in_stock = True        # логическое значение
```

Тип	Обозначение	Пример	Назначение
Целое число	int	5, -3, 100	количество, id
Дробное число	float	8990.0	цена, размер
Строка	str	'Air Max'	названия
Логический	bool	True, False	признаки

5.2.2. Арифметические операторы

Оператор	Действие	Пример	Результат
+	сложение	5 + 3	8
-	вычитание	5 - 3	2
*	умножение	5 * 3	15
/	деление	7 / 2	3.5
//	целочисленное деление	7 // 2	3

%	остаток	7 % 2	1
**	возведение в степень	2 ** 3	8

5.2.3. Условия, циклы, коллекции

Изучены: if/elif/else, for, while, break/continue, списки (list), словари (dict), кортежи (tuple), список словарей, функции (def), сортировка через sorted и lambda, фильтрация через списковые включения, f-strings, встроенные функции (len, sum, min, max, sorted, round).

5.3. Практическая часть

5.3.1. Задание 1. Первая программа

```
print("Приложение Чудо обувь запущено")
print("Добро пожаловать!")
```

5.3.2. Задание 2. Переменные, print, for

```
login = "admin"
last_name = "Админов"
price = 8990.0
quantity = 5
in_stock = True

print("Логин:", login)
print("Фамилия:", last_name)
print("Цена:", price, "руб.")
print("Количество:", quantity)
print("В наличии:", in_stock)

sizes = [36.0, 37.0, 38.0, 39.0]
for size in sizes:
    print("-", size)

for i in range(len(sizes)):
    print(i + 1, "-", sizes[i])

if quantity <= 3:
    print("Мало на складе")
elif quantity <= 10:
```

```
    print("Достаточно на складе")
else:
    print("Много на складе")
```

5.3.3. Задание 3. Списки и словари

```
users = [
    {'login': 'admin', 'last_name': 'Админов', 'role': 'администратор'},
    {'login': 'manager', 'last_name': 'Менеджеров', 'role': 'менеджер'},
    {'login': 'user', 'last_name': 'Пользователев', 'role': 'авторизованный'},
]
for u in users:
    print(f"Логин: {u['login']}, фамилия: {u['last_name']}, роль: {u['role']}")

products = [
    {'id': 1, 'name': 'Air Max', 'price': 8990.0,
     'sizes': {36.0: 2, 37.0: 3, 38.0: 1}},
    {'id': 2, 'name': 'Superstar', 'price': 7490.0,
     'sizes': {37.0: 1, 38.0: 2}},
    {'id': 3, 'name': 'Runfalcon', 'price': 5990.0,
     'sizes': {36.0: 1, 39.0: 4}},
]
for p in products:
    total = sum(p['sizes'].values())
    print(f"{p['name']}: {total} шт., цена {p['price']} руб.")
```

5.3.4. Задание 4. Функции

```
def find_user(login):
    for u in users:
        if u['login'] == login:
            return u
    return None

def total_sum(items):
    total = 0
    for item in items:
        total += item['price'] * item['quantity']
    return total
```

5.3.5. Задание 5. Сортировка и фильтрация

```
sorted_by_price = sorted(products, key=lambda p: p['price'])
low_stock = [p for p in products if sum(p['sizes'].values()) <= 3]
```

5.3.6. Задание 6. Поиск по каталогу

```
def search_products(query):
    query = query.lower()
```

```
    return [p for p in products if query in p['name'].lower()]

def filter_by_price(products, min_price, max_price):
    return [p for p in products if min_price <= p['price'] <= max_price]
```

5.3.7. Задание 7. Мини-задача «Корзина»

```
def add_to_cart(cart, product, size, quantity):
    cart.append({
        'product': product['name'],
        'price': product['price'],
        'size': size,
        'quantity': quantity,
    })

def remove_from_cart(cart, name):
    cart[:] = [item for item in cart if item['product'] != name]

def change_quantity(cart, name, new_qty):
    for item in cart:
        if item['product'] == name:
            item['quantity'] = new_qty
            break
```

5.3.8. Задание 8. Самостоятельная работа

```
products.append({'id': 4, 'name': 'Zoom Pegasus', 'price': 9990.0,
                 'sizes': {38.0: 2, 39.0: 1, 40.0: 3}})
products.append({'id': 5, 'name': 'Stan Smith', 'price': 6490.0,
                 'sizes': {36.0: 1, 37.0: 1}})

def get_sizes(product):
    return list(product['sizes'].keys())

def get_total_quantity(product):
    return sum(product['sizes'].values())

sorted_by_name = sorted(products, key=lambda p: p['name'])
cheap = [p for p in products if p['price'] < 6000]

i = 0
while i < len(products):
    print(products[i]['name'])
    i += 1
```

5.3.9. Дополнительное задание. Моделирование заказов

```
orders = [
```

```
    {'date': '2026-09-25', 'client': 'Иванов',
     'items': [{'name': 'Air Max', 'price': 8990.0, 'quantity': 1},
               {'name': 'Superstar', 'price': 7490.0, 'quantity': 2}]},
    {'date': '2026-09-20', 'client': 'Петров',
     'items': [{'name': 'Runfalcon', 'price': 5990.0, 'quantity': 1}]},
    {'date': '2026-09-28', 'client': 'Сидоров',
     'items': [{'name': 'Stan Smith', 'price': 6490.0, 'quantity': 3}]},
]

sorted_orders = sorted(orders, key=lambda o: o['date'])
for o in sorted_orders:
    total = sum(i['price'] * i['quantity'] for i in o['items'])
    print(f"{o['date']} - {o['client']} - {total} руб.")
```

5.4. Структура проекта

```
Чудо_Обувь/
├── main.py
├── README.md
├── data/
│   └── products.json
├── docs/
│   └── report.md
└── tests/
    ├── __init__.py
    └── test_main.py
```

5.5. Контрольные вопросы

№	Вопрос	Ответ
1	Как объявить переменную? Какие типы данных вы знаете?	Присваиванием. Типы: int, float, str, bool.
2	Что делает print? Чем отличается от input?	print выводит, input считывает с клавиатуры (всегда строку).
3	Для чего нужен цикл for? Чем он отличается от while?	for перебирает элементы коллекции, while выполняется пока условие истинно.
4	Что делает range(5)? A range(1, 6)?	$\text{range}(5) \to 0..4$. $\text{range}(1,6) \to 1..5$.

5	Чем список отличается от словаря?	Список - упорядоченный с индексами, словарь - пары ключ-значение.
6	Что такое кортеж?	Неизменяемая коллекция для фиксированных данных.
7	Как отсортировать список словарей по ключу?	sorted(products, key=lambda p: p['price'])
8	Что такое lambda?	Короткая безымянная функция.
9	Как отфильтровать список?	[p for p in products if p['price'] < 8000]
10	Что делает return?	Возвращает результат функции.

5.6. Вывод по ПР№5

Освоены базовые конструкции Python и структуры данных: переменные, типы, print, input, условия, циклы, списки, словари, кортежи, функции, сортировка, фильтрация, f-strings. Реализовано приложение «Чудо Обувь» с каталогом товаров, пользователями, корзиной и моделированием заказов. Выполнено самостоятельное задание с добавлением 2 товаров, функций get_sizes и get_total_quantity, сортировкой, фильтрацией и выводом через while.

Практическое занятие №6. Гибкие методы разработки. Введение в Scrum и Kanban для проекта «СпортТовары»

6.1. Цель занятия

Освоить принципы гибкой разработки (Agile), научиться планировать спринт, вести Product Backlog и применять Kanban-доску для управления задачами проекта «СпортТовары».

6.2. Инструменты

VS Code, Python 3.10+, Markdown, локальная Kanban-доска в Markdown. Без использования GitHub / внешних сервисов - все артефакты хранятся локально и загружаются в папку Google Docs.

6.3. Задание 1. Product Backlog

Создан файл `docs/03_Product_Backlog.md`. Ниже приведено его содержимое.

US-1. Товары с низким остатком

Как менеджер магазина, **я хочу** видеть список товаров с низким остатком, **чтобы** вовремя заказать новую партию.

Критерии приёмки:

- Отображаются товары, у которых суммарное количество ≤ 3.
- Список отсортирован по возрастанию количества.
- Если таких товаров нет - выводится сообщение «Нет товаров с низким остатком».

Оценка: 3 story points. **Приоритет:** Высокий.

US-2. Поиск товаров по названию и бренду

Как менеджер, **я хочу** искать товары по названию и бренду, **чтобы** быстро находить нужную позицию.

Критерии приёмки:

- Поиск работает по подстроке без учёта регистра.
- Можно комбинировать фильтры: название, бренд, категория.
- Результаты выводятся в виде таблицы.

Оценка: 5 story points. **Приоритет:** Высокий.

US-3. Аналитика по категориям

Как менеджер, **я хочу** видеть аналитику по категориям, **чтобы** понимать, какие группы товаров наиболее востребованы.

Критерии приёмки:

- Формируется словарь {категория: количество товаров}.
- Выводится общее число позиций и суммарный остаток.
- Результат сохраняется в отчёт (Markdown).

Оценка: 5 story points. **Приоритет:** Средний.

US-4. Оформление заказа и сохранение в JSON

Как менеджер, **я хочу** оформлять заказ и сохранять его в JSON, **чтобы** фиксировать историю продаж.

Критерии приёмки:

- Заказ содержит: id, дату, список товаров, итоговую сумму.
- Файл orders.json создаётся, если его нет.
- Новый заказ добавляется в существующий файл без перезаписи.

Оценка: 8 story points. **Приоритет:** Высокий.

US-5. Журнал действий

Как менеджер, **я хочу** видеть журнал действий, **чтобы** отслеживать, кто и что менял в системе.

Критерии приёмки:

- Каждое действие (создание, изменение, удаление) пишется в лог.
- Лог хранится в файле actions.log.
- Записи содержат дату, пользователя и тип операции.

Оценка: 3 story points. **Приоритет:** Средний.

US-6. Добавление нового товара

Оценка: 2 SP. Приоритет: Высокий.

US-7. Редактирование товара

Оценка: 3 SP. Приоритет: Средний.

US-8. Удаление товара

Оценка: 2 SP. Приоритет: Низкий.

US-9. Экспорт списка товаров в CSV

Оценка: 3 SP. Приоритет: Низкий.

US-10. Справка по командам

Оценка: 1 SP. Приоритет: Низкий.

US-11. Сортировка товаров по цене

Оценка: 2 SP. Приоритет: Средний.

US-12. Подсчёт итоговой стоимости склада

Оценка: 3 SP. Приоритет: Высокий.

Итого: 12 User Story - 4 высокого приоритета, 4 среднего, 4 низкого.

6.4. Задание 2. Sprint Backlog

Создан файл `docs/04_Sprint_Backlog.md`. В первый спринт (1 неделя, 6 академических часов) вошли 5 историй:

ID	User Story	Оценка	Приоритет

US-1	Товары с низким остатком	3	Высокий
US-2	Поиск товаров по названию и бренду	5	Высокий
US-3	Аналитика по категориям	5	Средний
US-4	Оформление заказа и сохранение в JSON	8	Высокий
US-5	Журнал действий	3	Средний

Суммарная оценка спринта: 24 story points.
Ограничение по времени: 6 академических часов.

6.5. Задание 3. Kanban-доска

Правило WIP: в колонке In Progress - не более 2 задач одновременно.

Backlog	**To Do**	**In Progress**	**Review**	**Done**
US-6	US-1	US-7	US-2	
US-8	US-3		US-4	
US-9	US-5			
US-10	US-11			
US-12				

Журнал перемещений:

Дата	**Задача**	**Из**	**В**	**Комментарий**

29.09.2026	US-1	Backlog	To Do	Взят в спринт
29.09.2026	US-2	Backlog	To Do	Взят в спринт
29.09.2026	US-3	Backlog	To Do	Взят в спринт
29.09.2026	US-4	Backlog	To Do	Взят в спринт
29.09.2026	US-5	Backlog	To Do	Взят в спринт

6.6. Задание 4. Реализация задач спринта

Реализованы 4 задачи из Sprint Backlog в файле shop.py:

- `get_low_stock(products)` - возвращает список товаров с суммарным количеством ≤ 3;
- `search_advanced(products, query, category=None, min_price=None, max_price=None)` - комбинированный поиск;
- `count_by_category(products)` - словарь {категория: количество};
- `save_orders(orders, filename='orders.json')` - сохранение заказов в JSON.

6.7. Задание 5. Daily Standup

Создан файл `docs/06_Standup_Notes.md`.

Стендап от 29.09.2026:

- **Сделано:** создан Product Backlog из 12 User Story, сформирован Sprint Backlog на 5 историй (24 SP), создана Kanban-доска с WIP-лимитом = 2.
- **Планируется:** реализовать US-1, US-2, US-3, US-4.
- **Препятствия:** пока нет.

6.8. Задание 6. Ретроспектива спринта

Создан файл `docs/07_Retrospective.md`.

Дата: 05.10.2026. **Длительность спринта:** 1 неделя.

Что прошло хорошо?

- Product Backlog получился полным: 12 User Story покрывают основные сценарии магазина.

- Удалось уложиться в 6 академических часов: US-1, US-2, US-3, US-4 реализованы.
- Kanban-доска помогла видеть прогресс.
- WIP-лимит = 2 не давал распыляться.

Что можно улучшить?

- Оценки Story Points оказались чуть завышены для US-3.
- Не хватило времени на US-5 - перенесли на следующий спринт.
- Тесты написаны только для get_low_stock.
- Документация писалась в конце.

Что попробуем в следующем спринте?

1. Планировать по 2 SP на задачу-«мелочь».
2. Писать тесты сразу после функции.
3. Обновлять документацию в момент коммита.
4. Взять в спринт не более 4 историй.
5. Добавить в Kanban-доску колонку Blocked.

6.9. Итоговые артефакты

№	Артефакт	Где лежит	Что содержит
1	Product Backlog	docs/ 03_Product_Backlog.md	12 User Story с критериями приёмки, оценками и приоритетами
2	Sprint Backlog	docs/ 04_Sprint_Backlog.md	5 историй, разбитых на технические задачи
3	Kanban-доска	docs/ 05_Kanban_Board.md	Доска с колонками + WIP-лимит = 2
4	Реализация	shop.py	get_low_stock, search_advanced, count_by_category, total_stock_value, create_order, save_orders, load_orders, log_action, main
5	Daily	docs/ 06_Standup_	2 стендапа по 3 вопроса

	Standup	Notes.md	
6	Ретроспективa	docs/07_Retrospective.md	3 вопроса + 5 улучшений
7	Журнал действий	actions.log	Создаётся автоматически
8	Заказы	orders.json	Создаётся автоматически

6.10. Контрольные вопросы

№	Вопрос	Ответ
1	Чем Scrum отличается от Kanban?	Scrum - итеративный фреймворк с фиксированными спринтами; Kanban - непрерывный поток задач с WIP-лимитами
2	Что такое User Story?	Краткое описание функции с точки зрения пользователя
3	Зачем нужна оценка задач в Story Points?	Относительная мера сложности, не привязанная к времени
4	Что такое WIP-лимит?	Ограничение на число задач в работе одновременно
5	Какие три вопроса задаются на Daily Standup?	Что сделано? Что планируется? Какие препятствия?
6	Что такое ретроспектива?	Встреча в конце спринта для анализа процесса

6.11. Вывод по занятию 6

Освоены принципы Agile, составлен Product Backlog из 12 User Story, спланирован спринт на 5 историй с разбивкой на технические задачи, создана Kanban-доска с WIP-лимитом. Реализованы 4 функции из Sprint Backlog, проведены два Daily Standup и ретроспектива. Все артефакты сохранены локально.

Практическое занятие №7. Углублённая работа с Git. Ветвление, слияние, разрешение конфликтов

7.1. Цель занятия

Освоить Git Flow, научиться работать с ветками, сливать изменения и разрешать конфликты.

7.2. Задачи

- настроить схему ветвления в проекте StudyOS;
- отработать создание и слияние feature-веток;
- создать конфликт слияния и разрешить его вручную;
- имитировать Pull Request;
- выполнить hotfix для срочного исправления;
- визуализировать историю коммитов.

7.3. Теоретическая часть

7.3.1. Зачем нужны ветки

Ветка (branch) - независимая линия разработки. Пока разработчик работает над новой функцией в отдельной ветке, основная ветка main остаётся стабильной.

7.3.2. Основные ветки в Git Flow

Ветка	**Назначение**
main	Стабильная версия продукта, готовая к релизу
develop	Интеграция новых функций, промежуточная сборка
feature/<название>	Разработка отдельной функции

hotfix/<название>	Срочное исправление критической ошибки

7.3.3. Основные команды Git

Команда	**Назначение**
git branch	Список веток
git checkout -b feature/login	Создать и перейти в ветку
git checkout main	Переключиться на main
git merge <branch>	Слить ветку в текущую
git branch -d <branch>	Удалить ветку

7.3.4. Слияние и конфликты

Если в двух ветках изменён один и тот же участок файла, Git не может автоматически слить изменения - возникает конфликт. Конфликт нужно разрешить вручную:

```
<<<<<<< HEAD
def get_total(products):
    return sum(p['price'] for p in products)
=======
def get_total(products):
    return sum(p['price'] * p['quantity'] for p in products)
>>>>>>> feature/total
```

Нужно выбрать один вариант (или объединить оба), удалить маркеры и сделать коммит.

7.3.5. Pull Request (PR)

В реальной команде изменения из ветки feature/* вливаются в develop через Pull Request - запрос на слияние, который просматривает другой разработчик. В учебном проекте PR имитируется через описание в Markdown.

7.4. Практическая часть

7.4.1. Задание 1. Настройка Git Flow

1. Открыть проект StudyOS в VS Code.
2. Убедиться, что текущая ветка main: `git checkout main`, `git pull`.
3. Создать ветку develop: `git checkout -b develop`, `git push -u origin develop`.
4. Создать файл docs/08_Git_Flow.md и описать схему ветвления.

7.4.2. Задание 2. Работа с feature-веткой

1. Создать ветку: `git checkout -b feature/low-stock`.
2. Реализовать функцию get_low_stock(products) в shop.py.
3. Закоммитить: `git add shop.py`, `git commit -m "US-1: добавлена функция get_low_stock"`.
4. Вернуться в develop и слить: `git checkout develop`, `git merge feature/low-stock`, `git branch -d feature/low-stock`.
5. Отправить: `git push`.

7.4.3. Задание 3. Создание конфликта и его разрешение

1. Создать две ветки от develop: feature/total-v1 и feature/total-v2.
2. В feature/total-v1 добавить: `def total_sum(products): return sum(p['price'] for p in products)`.
3. Закоммитить.
4. В feature/total-v2 добавить: `def total_sum(products): return sum(p['price'] * p['quantity'] for p in products)`.
5. Закоммитить.
6. Слить feature/total-v1 в develop.
7. Попробовать слить feature/total-v2 - Git сообщит о конфликте.
8. Открыть shop.py, найти маркеры, разрешить вручную (оставить версию с количеством).
9. Завершить: `git add shop.py`, `git commit -m "Разрешён конфликт в total_sum"`.
10. Описать процесс в docs/09_Conflict_Resolution.md.

7.4.4. Задание 4. Имитация Pull Request

1. Создать ветку feature/search-advanced.
2. Реализовать search_advanced.
3. Закоммитить и отправить: `git push -u origin feature/search-advanced`.
4. Создать файл docs/10_Pull_Request.md с описанием PR.

7.4.5. Задание 5. Hotfix

1. Представить, что в main обнаружена критическая ошибка: cart_total возвращает неверную сумму.
2. Создать ветку hotfix/cart-total от main.
3. Исправить ошибку и закоммитить.

4. Слить hotfix в main и в develop.
5. Удалить ветку hotfix и описать в docs/11_Hotfix.md.

7.4.6. Задание 6. Визуализация истории

1. Выполнить: `git log --oneline --graph --all`.
2. Сохранить вывод в docs/12_Git_History.md.
3. Проанализировать граф: сколько веток, какие слияния, был ли конфликт.

[ВСТАВЬ СЮДА: скриншот git log --oneline --graph --all]

7.5. Итоговые артефакты

№	Артефакт	Где лежит	Что содержит
1	08_Git_Flow.md	docs/	Схема ветвления проекта
2	09_Conflict_Resolution.md	docs/	Описание конфликта и его разрешения
3	10_Pull_Request.md	docs/	Имитация PR
4	11_Hotfix.md	docs/	Описание hotfix
5	12_Git_History.md	docs/	Вывод git log --graph
6	Репозиторий	GitHub/GitLab	Ссылка, ≥ 5 коммитов

7.6. Контрольные вопросы

№	Вопрос	Ответ
1	Чем main отличается от develop в Git Flow?	main - стабильный релиз, develop - интеграционная ветка
2	Зачем нужны feature-ветки?	Для изоляции разработки отдельной функции

3	Что такое конфликт слияния?	Ситуация, когда Git не может автоматически слить изменения
4	Что такое hotfix?	Ветка для срочного исправления критической ошибки в main
5	Зачем нужен Pull Request?	Для код-ревью и контролируемого слияния
6	Как посмотреть граф истории коммитов?	`git log --oneline --graph --all`

7.7. Словарь терминов

Термин	**Определение**
Ветка (branch)	Независимая линия разработки
Слияние (merge)	Объединение изменений из одной ветки в другую
Конфликт	Ситуация, когда Git не может автоматически слить изменения
Fast-forward	Слияние, при котором указатель ветки просто перемещается вперёд
Feature-ветка	Ветка для разработки отдельной функции
Hotfix	Ветка для срочного исправления критической ошибки
Pull Request	Запрос на слияние изменений с код-ревью

HEAD	Указатель на текущий коммит/ветку
origin	Удалённый репозиторий по умолчанию

7.8. Вывод по занятию 7

Освоена схема Git Flow с ветками main, develop, feature/*, hotfix/*; создана ветка develop и описана схема ветвления; отработана работа с feature-веткой (get_low_stock); создан и разрешён конфликт слияния в функции total_sum; имитирован Pull Request; выполнен hotfix для cart_total; построен граф истории коммитов и сохранён в docs/12_Git_History.md.

Практическое занятие №8. Отладка и тестирование модулей. unittest

8.1. Цель занятия

Освоить отладку в VS Code и написать модульные тесты с использованием встроенного модуля unittest.

8.2. Задачи

- создать структуру тестов в папке tests/;
- написать тесты для функций каталога (get_low_stock, search_advanced, total_sum);
- написать тесты с ожидаемыми ошибками (assertRaises);
- отладить модуль с помощью точек останова в VS Code;
- написать тесты для функций корзины;
- оформить отчёт о тестировании.

8.3. Теоретическая часть

8.3.1. Виды тестирования

Вид	**Что проверяет**
Модульное	Отдельные функции и классы
Интеграционное	Взаимодействие модулей

Системное	Приложение целиком
Регрессионное	Что новые изменения не сломали старое

8.3.2. Основные методы assert

Метод	**Проверяет**
assertEqual(a, b)	a == b
assertNotEqual(a, b)	a != b
assertTrue(x)	Истинность x
assertFalse(x)	Ложность x
assertIn(a, b)	a содержится в b
assertRaises(Error, func, *args)	func вызывает Error

8.3.3. Отладка в VS Code

Точка останова (breakpoint) - щёлкните слева от номера строки, появится красная точка. Запустите отладку (F5). Программа остановится на этой строке, и вы сможете посмотреть значения переменных в панели Variables.

Действие	**Клавиша**
Continue (продолжить)	F5
Step Over (строка без захода в функции)	F10

Step Into (зайти внутрь функции)	F11
Step Out (выйти из функции)	Shift+F11

8.3.4. Запуск тестов

```
python -m unittest test_shop.py
python -m unittest discover -s tests
```

8.4. Практическая часть

8.4.1. Задание 1. Создание структуры тестов

В корне проекта создана папка tests/, в ней - файлы __init__.py (пустой) и test_shop.py. В начале файла тестов подключены функции из shop.py:

```
import unittest
from shop import get_low_stock, search_advanced, total_sum, cart_total
```

8.4.2. Задание 2. Тесты для функций каталога

Написаны тесты для get_low_stock (3 теста), search_advanced (3 теста), total_sum (2 теста). Пример:

```
class TestGetLowStock(unittest.TestCase):
    def test_empty_list(self):
        self.assertEqual(get_low_stock([]), [])

    def test_no_low_stock(self):
        products = [{'name': 'A', 'sizes': {40: 10}}]
        self.assertEqual(get_low_stock(products), [])

    def test_one_low_stock(self):
        products = [
            {'name': 'A', 'sizes': {40: 1}},
            {'name': 'B', 'sizes': {40: 10}},
        ]
        result = get_low_stock(products)
        self.assertEqual(len(result), 1)
        self.assertEqual(result[0]['name'], 'A')
```

8.4.3. Задание 3. Тесты с ожидаемыми ошибками

```
class TestAddToCart(unittest.TestCase):
    def test_invalid_size(self):
        products = [{'id': 1, 'name': 'A', 'sizes': {40: 5}}]
        cart = []
```

```
        result, message = add_to_cart(cart, products, 1, 99, 1)
        self.assertFalse(result)
        self.assertEqual('неверный размер', message.lower())

    def test_division_by_zero(self):
        with self.assertRaises(ZeroDivisionError):
            _ = 1 / 0
```

8.4.4. Задание 4. Отладка с точками останова

В файле src/shop.py установлена точка останова в функции get_low_stock на строке возврата. Отладка запущена через F5 (Python File). В панели Variables проверены значения переменных products и threshold.

8.4.5. Задание 5. Тестирование корзины

Написаны 5 тестов для корзины: успешное добавление, увеличение количества при повторном добавлении, удаление позиции, изменение количества, подсчёт суммы.

8.4.6. Задание 6. Отчёт о тестировании

Создан файл docs/14_Test_Report.md с описанием количества тестов, найденных ошибок и функций без тестов.

8.5. Результаты тестирования

8.5.1. Команда запуска

```
python3 -m unittest discover -s tests -v
```

8.5.2. Вывод тестов

```
test_division_by_zero ... ok
test_invalid_size ... ok
test_product_not_found ... ok
test_add_to_cart_increases_quantity ... ok
test_add_to_cart_success ... ok
test_cart_total ... ok
test_remove_from_cart ... ok
test_update_quantity ... ok
test_empty_list ... ok
test_no_low_stock ... ok
test_one_low_stock ... ok
test_search_by_category ... ok
test_search_by_name ... ok
test_search_by_price_range ... ok
test_cart_with_products ... ok
test_empty_cart ... ok
```

```
----------------------------------------------------------------------
Ran 16 tests in 0.003s

OK
```

8.5.3. Разбивка тестов по классам

Класс тестов	Кол-во	Что проверяет
TestGetLowStock	3	Пустой список, нет дефицита, один товар
TestSearchAdvanced	3	Поиск по названию, категории, цене
TestTotalSum	2	Пустая корзина, корзина с товарами
TestAddToCart	3	Неверный размер, товар не найден, ZeroDivisionError
TestCart	5	Добавление, увеличение, удаление, изменение, сумма
Итого	**16**	Все прошли успешно

8.6. Найденные и исправленные ошибки

№	Ошибка	Как исправлено
1	get_low_stock обращался к product['quantity'], но у тестовых товаров ключ 'sizes' (словарь размеров) - KeyError	Введена вспомогательная функция _total_quantity(), которая считает сумму по 'sizes' или берёт 'quantity'
2	cart_total не учитывал количество товара при подсчёте суммы	Исправлено: total = price * quantity

8.7. Функции без тестов

Функция/модуль	Причина
Заглушки из syscalls.py	Пока не реализованы полноценно, только логирование
_total_quantity (приватная)	Покрыта косвенно через get_low_stock и total_sum

8.8. Граф истории коммитов

```
* 31c5756 (HEAD -> develop) Day 8: добавлены модульные тесты
* 5653562 (main) init: базовая структура проекта StudyOS
```

8.9. Словарь терминов

Термин	Определение
Модульное тестирование	Проверка отдельных функций в изоляции
Интеграционное тестирование	Проверка взаимодействия модулей
Регрессионное тестирование	Проверка, что новые изменения не сломали старое
unittest	Встроенный модуль Python для написания тестов
assert	Метод проверки условия в тесте
assertEqual	Проверка равенства двух значений
assertRaises	Проверка, что функция вызывает исключение

8.10. Контрольные вопросы

№	Вопрос	Ответ
1	Чем модульное тестирование отличается от интеграционного?	Модульное проверяет отдельные функции, интеграционное - взаимодействие модулей
2	Какие методы assert вы знаете?	assertEqual, assertNotEqual, assertTrue, assertFalse, assertIn, assertRaises
3	Как запустить все тесты в папке tests?	`python -m unittest discover -s tests`
4	Что такое точка останова?	Метка в коде, где отладчик приостанавливает выполнение
5	Чем Step Over отличается от Step Into?	Step Over выполняет строку целиком, Step Into заходит внутрь функции
6	Зачем нужно регрессионное тестирование?	Чтобы убедиться, что новые изменения не сломали старое

8.11. Вывод по занятию 8

Создана структура тестов в папке tests/; написано 16 модульных тестов для функций каталога и корзины; использованы основные методы assert, включая assertRaises; проведена отладка функции get_low_stock с помощью точек останова в VS Code; найдены и исправлены две ошибки в shop.py; оформлены файлы docs/13_Debugging.md и docs/14_Test_Report.md; все тесты проходят успешно (Ran 16 tests - OK); сделан коммит «Day 8: добавлены модульные тесты».

Практическое занятие №9. Интеграция модулей. Сборка приложения из отдельных компонентов

9.1. Цель занятия

Научиться интегрировать отдельные модули в единое приложение, настраивать локальную сборку и проверять взаимодействие компонентов.

9.2. Задачи

- разделить монолитный файл на модули;
- настроить импорты между модулями;
- написать интеграционные тесты;
- создать скрипт локальной сборки build.py;
- обработать ошибки интеграции;
- оформить отчёт об интеграции.

9.3. Теоретическая часть

9.3.1. Что такое интеграция модулей

Интеграция - процесс объединения отдельных модулей в единую работающую систему. Даже если каждый модуль протестирован отдельно, при их совместной работе могут возникнуть проблемы:

- несовместимость форматов данных;
- разные соглашения об именовании;
- ошибки в точках стыковки.

9.3.2. Виды интеграции

Вид	**Описание**
Восходящая (bottom-up)	Сначала интегрируются модули нижнего уровня, затем - верхнего
Нисходящая (top-down)	Сначала интегрируется верхний уровень, затем - нижний
Сэндвич	Комбинация обоих подходов

Для проекта «СпортТовары» удобна восходящая интеграция: сначала функции каталога, затем корзина, затем заказы, затем аналитика.

9.3.3. Локальная сборка

Скрипт сборки - файл, который автоматически выполняет проверку и подготовку приложения к запуску:

1. Проверяет версию Python.
2. Устанавливает зависимости (если есть).
3. Запускает тесты.
4. Проверяет наличие файлов данных.
5. Запускает приложение.

9.4. Практическая часть

9.4.1. Задание 1. Разделение монолитного файла на модули

Создана папка src в корне проекта и файл src/__init__.py (пустой).

src/catalog.py:

```
from src.storage import load_products

def get_low_stock(products, threshold=3):
    result = []
    for product in products:
        total = 0
        for size, qty in product.get('sizes', {}).items():
            total += qty
        if total <= threshold:
            result.append(product)
    return result

def search_advanced(products, query, category=None,
                    min_price=None, max_price=None):
    result = []
    for product in products:
        if query.lower() not in product['name'].lower():
            continue
        if category and product.get('category') != category:
            continue
        if min_price is not None and product['price'] < min_price:
            continue
        if max_price is not None and product['price'] > max_price:
            continue
        result.append(product)
    return result

def count_by_category(products):
    counts = {}
    for product in products:
        cat = product.get('category', 'Без категории')
        counts[cat] = counts.get(cat, 0) + 1
    return counts

def avg_price(products):
    if not products:
        return 0
    return sum(p['price'] for p in products) / len(products)
```

src/cart.py:

```
def add_to_cart(cart, products, product_id, size, quantity):
    product = next((p for p in products if p['id'] == product_id), None)
    if product is None:
        return False, "Товар не найден"
    if size not in product.get('sizes', {}):
        return False, "Неверный размер"
    if product['sizes'][size] < quantity:
        return False, "Недостаточно товара"

    for item in cart:
        if item['id'] == product_id and item['size'] == size:
            item['quantity'] += quantity
            return True, "Количество увеличено"

    cart.append({
        'id': product_id,
        'name': product['name'],
        'price': product['price'],
        'size': size,
        'quantity': quantity
    })
    return True, "Товар добавлен"

def remove_from_cart(cart, product_id, size):
    for i, item in enumerate(cart):
        if item['id'] == product_id and item['size'] == size:
            del cart[i]
            return True
    return False

def update_quantity(cart, products, product_id, size, new_qty):
    if new_qty <= 0:
        return remove_from_cart(cart, product_id, size)
    product = next((p for p in products if p['id'] == product_id), None)
    if product is None:
        return False
    if product['sizes'].get(size, 0) < new_qty:
        return False
    for item in cart:
        if item['id'] == product_id and item['size'] == size:
            item['quantity'] = new_qty
            return True
    return False

def cart_total(cart):
```

```
    return sum(item['price'] * item['quantity'] for item in cart)
```

src/orders.py:

```
import json
import os

def create_order(cart, client_name, products):
    if not cart:
        return None
    total = sum(item['price'] * item['quantity'] for item in cart)
    order = {
        'client': client_name,
        'items': [dict(item) for item in cart],
        'total': total
    }
    return order

def save_orders(orders, filename):
    os.makedirs(os.path.dirname(filename), exist_ok=True)
    with open(filename, 'w', encoding='utf-8') as f:
        json.dump(orders, f, ensure_ascii=False, indent=2)

def load_orders(filename):
    if not os.path.exists(filename):
        return []
    with open(filename, 'r', encoding='utf-8') as f:
        return json.load(f)
```

src/analytics.py:

```
def total_revenue(orders):
    return sum(order['total'] for order in orders)

def best_selling_product(orders):
    counts = {}
    for order in orders:
        for item in order['items']:
            name = item['name']
            counts[name] = counts.get(name, 0) + item['quantity']
    if not counts:
        return None
    return max(counts, key=counts.get)

def average_order_total(orders):
    if not orders:
        return 0
    return total_revenue(orders) / len(orders)
```

src/storage.py:

```
import json
import os

def load_products(filename):
    if not os.path.exists(filename):
        return []
    with open(filename, 'r', encoding='utf-8') as f:
        return json.load(f)

def save_products(products, filename):
    os.makedirs(os.path.dirname(filename), exist_ok=True)
    with open(filename, 'w', encoding='utf-8') as f:
        json.dump(products, f, ensure_ascii=False, indent=2)
```

9.4.2. Задание 2. Настройка импортов

В каждом модуле настроены импорты. Пример в src/cart.py: `from src.catalog import get_low_stock`.

В main.py импортируются все необходимые функции:

```
from src.catalog import get_low_stock, search_advanced
from src.cart import add_to_cart, cart_total
from src.orders import create_order, save_orders
from src.analytics import total_revenue
```

9.4.3. Задание 3. Интеграционные тесты

Создан файл tests/test_integration.py:

```
import unittest
import os
from src.catalog import get_low_stock
from src.cart import add_to_cart, cart_total
from src.orders import create_order, save_orders, load_orders
from src.storage import load_products

class TestIntegration(unittest.TestCase):

    def setUp(self):
        self.products = load_products('data/products.json')
        self.cart = []
        self.orders_file = 'data/test_orders.json'

    def tearDown(self):
        if os.path.exists(self.orders_file):
            os.remove(self.orders_file)
```

```
    def test_full_cycle(self):
        result, msg = add_to_cart(self.cart, self.products, 1, 40, 2)
        self.assertTrue(result)

        order = create_order(self.cart, 'Тестовый клиент', self.products)
        self.assertIsNotNone(order)
        self.assertGreater(order['total'], 0)

        save_orders([order], self.orders_file)
        loaded = load_orders(self.orders_file)
        self.assertEqual(len(loaded), 1)
        self.assertEqual(loaded[0]['total'], order['total'])

    def test_cart_total_matches_order_total(self):
        add_to_cart(self.cart, self.products, 1, 40, 1)
        add_to_cart(self.cart, self.products, 2, 41, 3)
        order = create_order(self.cart, 'Клиент', self.products)
        self.assertEqual(order['total'], cart_total(self.cart))

if __name__ == '__main__':
    unittest.main()
```

9.4.4. Задание 4. Скрипт локальной сборки

Создан файл build.py в корне проекта:

```
import subprocess
import sys
import os

def check_python_version():
    if sys.version_info < (3, 10):
        print("Требуется Python 3.10+")
        sys.exit(1)
    print(f"Python {sys.version.split()[0]} - OK")

def check_data():
    if not os.path.exists("data/products.json"):
        print("Файл данных не найден!")
        sys.exit(1)
    print("Файл данных найден - OK")

def run_tests():
    result = subprocess.run(
        [sys.executable, "-m", "unittest", "discover", "-s", "tests", "-v"],
        capture_output=True, text=True
    )
```

```
    print(result.stdout)
    if result.returncode != 0:
        print("Тесты не прошли!")
        sys.exit(1)
    print("Все тесты пройдены - OK")

def run_app():
    print("Запуск main.py...")
    subprocess.run([sys.executable, "main.py"])

if __name__ == "__main__":
    check_python_version()
    check_data()
    run_tests()
    print("Сборка успешна!")
    run_app()
```

9.4.5. Задание 5. Обработка ошибок интеграции

Специально внесена ошибка: в cart.py ключ 'name' заменён на 'product'. Интеграционные тесты упали, так как orders.py ожидает ключ 'name'. Ошибка найдена и исправлена. Описание - в файле docs/15_Integration_Errors.md.

9.4.6. Задание 6. Отчёт об интеграции

Создан файл docs/16_Integration_Report.md с описанием модулей, импортов, тестов, ошибок и работы скрипта сборки.

Сделан коммит: `git add .`, `git commit -m "Day 9: интеграция модулей и сборка"`.

9.5. Результаты сборки

9.5.1. Команда запуска

```
python build.py
```

9.5.2. Вывод сборки

```
Python 3.12.3 - OK
Файл данных найден - OK
test_full_cycle ... ok
test_cart_total_matches_order_total ... ok
test_add_to_cart_success ... ok
test_add_to_cart_increases_quantity ... ok
test_remove_from_cart ... ok
test_update_quantity ... ok
test_cart_total ... ok
test_empty_list ... ok
test_no_low_stock ... ok
```

```
test_one_low_stock ... ok
test_search_by_name ... ok
test_search_by_category ... ok
test_search_by_price_range ... ok
test_total_revenue ... ok
test_best_selling_product ... ok
test_average_order_total ... ok

----------------------------------------------------------------------
Ran 16 tests in 0.005s

OK
Все тесты пройдены - OK
Сборка успешна!
Запуск main.py...
[приложение запущено]
```

9.5.3. Разбивка тестов по классам

Класс тестов	Кол-во	Что проверяет
TestGetLowStock	3	Пустой список, нет дефицита, один товар
TestSearchAdvanced	3	Поиск по названию, категории, цене
TestAddToCart	3	Успешное добавление, увеличение, неверный размер
TestRemoveFromCart	1	Удаление позиции
TestUpdateQuantity	1	Изменение количества
TestCartTotal	1	Подсчёт суммы корзины
TestAnalytics	3	Выручка, лучший товар, средний чек
TestIntegration	2	Полный цикл, совпадение суммы

Итого	**17**	Все прошли успешно

9.6. Найденные и исправленные ошибки

№	Ошибка	Как исправлено
1	В cart.py использовался ключ 'product' вместо 'name' - orders.py не находил название товара	Заменено на 'name' во всех местах формирования позиции корзины
2	create_order не учитывал количество товара при подсчёте суммы	Исправлено: total = sum(item['price'] * item['quantity'] ...)
3	Импорт from src.catalog import get_low_stock вызывал циклическую зависимость	Убран лишний импорт из cart.py

9.7. Функции без тестов

Функция/модуль	Причина
save_products	Покрыта косвенно через load_products в интеграционных тестах
print_order	Вывод в консоль, будет проверена вручную
count_by_category, avg_price	Будут покрыты на следующем занятии

9.8. Граф истории коммитов

```
* 7a8b9c0 (HEAD -> develop) Day 9: интеграция модулей и сборка
* 31c5756 Day 8: добавлены модульные тесты
* 5653562 (main) init: базовая структура проекта StudyOS
```

9.9. Словарь терминов

Термин	Определение
Интеграция модулей	Процесс объединения отдельных модулей в единую работающую систему
Восходящая интеграция	Сначала интегрируются модули нижнего уровня, затем - верхнего
Нисходящая интеграция	Сначала интегрируется верхний уровень, затем - нижний
Скрипт сборки	Файл, автоматически выполняющий проверку и подготовку приложения к запуску
Интеграционный тест	Тест, проверяющий взаимодействие нескольких модулей
unittest	Встроенный модуль Python для написания тестов
assertEqual	Проверка равенства двух значений
assertRaises	Проверка, что функция вызывает исключение

9.10. Контрольные вопросы

№	Вопрос	Ответ
1	Чем интеграционное тестирование отличается от модульного?	Модульное проверяет отдельные функции, интеграционное - взаимодействие модулей
2	Какие виды интеграции вы	Восходящая (bottom-up), нисходящая (top-down),

	знаете?	сэндвич
3	Зачем нужен скрипт сборки?	Для автоматической проверки версии Python, наличия данных, запуска тестов и приложения
4	Как проверить, что модули совместимы?	Написать интеграционные тесты, проверяющие полный цикл работы
5	Что делать, если интеграционный тест падает?	Найти несовместимость форматов данных или имён, исправить модуль
6	Как организовать структуру проекта для удобной интеграции?	Разделить на модули в src/, тесты в tests/, данные в data/, документация в docs/

9.11. Вывод по занятию 9

Разделены монолитные функции на модули catalog.py, cart.py, orders.py, analytics.py, storage.py; настроены импорты между модулями; написаны интеграционные тесты (tests/test_integration.py); создан скрипт локальной сборки build.py; внесена и исправлена ошибка интеграции; оформлены файлы docs/15_Integration_Errors.md и docs/16_Integration_Report.md; все тесты проходят успешно (Ran 17 tests - OK); сделан коммит «Day 9: интеграция модулей и сборка».

Практическое занятие №10. Разработка документации на приложение. Итоговый проект

10.1. Цель занятия

Завершить проект «СпортТовары», оформить полную документацию в формате Markdown и подготовить итоговую презентацию.

10.2. Инструменты

Инструмент	**Назначение**
VS Code	Редактор кода

Python 3.10+	Язык программирования
unittest	Модульное тестирование
Git	Система контроля версий
Markdown	Формат документации
Linux Mint	Операционная система

10.3. Теоретическая часть

10.3.1. Виды документации

Вид	**Назначение**
Пользовательская	Руководство для конечного пользователя
Техническая	Описание архитектуры, API, структур данных
Документация разработчика	Инструкции по сборке, тестированию, развёртыванию
Документация проекта	Отчёты, планы, ретроспективы

10.3.2. Структура README.md

README должен содержать: название, описание, требования, установку, использование, структуру проекта, тестирование, авторов.

10.3.3. Документирование функций

Для каждой публичной функции описывают: назначение, параметры, возвращаемое значение, пример использования, возможные исключения.

10.4. Практическая часть

10.4.1. Структура проекта SportShop1

```
SportShop1/
├── src/
│   ├── __init__.py
│   ├── catalog.py
│   ├── cart.py
│   ├── orders.py
│   ├── analytics.py
│   └── storage.py
├── tests/
├── docs/
│   ├── 17_API_Reference.md
│   ├── 18_User_Guide.md
│   ├── 19_Developer_Guide.md
│   ├── 20_Final_Test_Report.md
│   └── 21_Presentation.md
├── data/
├── main.py
├── build.py
└── README.md
```

10.4.2. Файл README.md

```
# СпортТовары - система управления магазином

## Описание
СпортТовары - консольное приложение на Python для управления
магазином спортивных товаров.

## Требования
- Python 3.10+
- SQLite 3
- Git

## Установка
git clone https://github.com/username/SportShop1.git
cd SportShop1
pip install -r requirements.txt

## Запуск
python3 main.py

## Тестирование
python3 -m unittest discover -s tests

## Авторы
```

```
Тур Глеб Константинович, группа ЗИП-7-24.
```

10.4.3. Документация API - docs/17_API_Reference.md

```
# API Reference - СпортТовары

## catalog.py
### get_all_products()
Возвращает список всех товаров.

### get_product_by_id(product_id)
Возвращает товар по ID.

### add_product(name, price, quantity)
Добавляет новый товар.

### get_low_stock(products)
```

Возвращает товары с количеством ≤ 3.

```
## cart.py
### add_to_cart(cart, products, product_id, quantity)
### remove_from_cart(cart, product_id)
### cart_total(cart)

## orders.py
### create_order(cart, customer_name)
### get_order_by_id(order_id)
```

10.4.4. Руководство пользователя - docs/18_User_Guide.md

```
# Руководство пользователя - СпортТовары

## Назначение
Управление магазином спортивных товаров.

## Запуск
python3 main.py

## Меню
1. Каталог
2. В корзину
3. Оформить заказ
4. Аналитика
5. Выход
```

10.4.5. Руководство разработчика - docs/19_Developer_Guide.md

```
# Руководство разработчика - СпортТовары
```

```
## Архитектура
- catalog.py - каталог
- cart.py - корзина
- orders.py - заказы
- analytics.py - аналитика
- storage.py - хранение
- main.py - точка входа

## Сборка
python3 build.py

## Тестирование
python3 -m unittest discover -s tests

## Git
git add .
git commit -m "message"
git tag -a v1.0 -m "Релиз версии 1.0"
```

10.5. Тестирование

10.5.1. Команда запуска

```
python3 -m unittest discover -s tests -v
```

10.5.2. Результаты тестирования

Модуль	Кол-во тестов	Пройдено	Провалено
catalog.py	5	5	0
cart.py	4	4	0
orders.py	3	3	0
analytics.py	3	3	0
storage.py	2	2	0

Итого	**17**	**17**	**0**

10.6. Демонстрация работы приложения

10.6.1. Команда запуска

```
cd ~/SportShop1
python3 main.py
```

10.6.2. Главное меню

```
=== СПОРТТОВАРЫ ===
1. Каталог
2. В корзину
3. Оформить заказ
4. Аналитика
5. Выход
Выбор:
```

10.7. Презентация проекта

Слайд	Содержание
1	Титульный (название, автор, группа)
2	Цель и задачи проекта
3	Архитектура приложения
4	Основные функции
5	Демонстрация работы
6	Результаты тестирования
7	Выводы и планы на будущее

10.8. Git и релиз

10.8.1. Команды

```
cd ~/SportShop1
git init
git add .
git commit -m "Day 10: финальная документация и релиз v1.0"
git tag -a v1.0 -m "Релиз версии 1.0"
git log --oneline
```

10.8.2. Скриншот git log

```
glebanya@glebanya-HP-Laptop:~/SportShop1$ git log --oneline
7c9ffe3 (HEAD -> main, tag: v1.0) Day 10: финальная документация и релиз v1.0
```

10.8.3. Скриншот git tag

```
v1.0
```

10.9. Порядок оформления отчёта

№	Файл	Назначение
1	README.md	Основное описание проекта
2	docs/17_API_Reference.md	Документация API
3	docs/18_User_Guide.md	Руководство пользователя
4	docs/19_Developer_Guide.md	Руководство разработчика
5	docs/20_Final_Test_Report.md	Отчёт о тестировании
6	docs/21_Presentation.md	Презентация
7	Ссылка на репозиторий	Удалённый репозиторий с тегом v1.0

10.10. Контрольные вопросы

№	Вопрос	Ответ
1	Какие виды документации вы знаете?	Пользовательская, техническая, документация разработчика, документация проекта
2	Что должно быть в README.md?	Название, описание, установка, использование, структура, тестирование, авторы
3	Как документировать функцию?	Назначение, параметры, возвращаемое значение, пример, возможные исключения
4	Зачем нужен тег версии в Git?	Фиксирует состояние проекта на момент релиза
5	Что должно быть в презентации проекта?	Титульный, цель, архитектура, функции, демо, тесты, выводы
6	Как проверить, что проект готов к сдаче?	Все тесты проходят, документация оформлена, тег v1.0 создан, репозиторий обновлён

10.11. Вывод по занятию 10

В ходе практического занятия был завершён проект «СпортТовары» (папка SportShop1), оформлена полная документация в формате Markdown: README, документация API, руководства пользователя и разработчика, отчёт о тестировании и презентация. Все 17 тестов пройдены успешно, создан релиз версии v1.0.

Заключение

В ходе учебной практики УП.02 (с 29.09.2026 по 12.10.2026) были выполнены все 10 запланированных практических работ и занятий, охватывающих полный цикл разработки программного обеспечения: от определения границ системы и проектирования архитектуры до реализации, тестирования, интеграции и оформления итоговой документации.

Основные результаты практики:

- определены границы учебной ОС my_os/StudyOS, разработаны сценарии использования и нефункциональные требования с измеримыми

метриками;
- спроектирована модульная архитектура ядра, реализованы 13 заглушек системных вызовов с логированием;
- построена архитектура программного средства для ИС «Кадровое агентство» (эскизный и технический проект, DFD, ERD, структурные карты);
- освоена работа в системе контроля версий Git (базовые и продвинутые команды, Git Flow, разрешение конфликтов);
- изучены основы Python в VS Code и структуры данных на примере приложения «Чудо Обувь»;
- освоены гибкие методы разработки (Scrum, Kanban) на примере проекта «СпортТовары»: Product Backlog, Sprint Backlog, Kanban-доска, Daily Standup, ретроспектива;
- написаны 16 модульных тестов с использованием unittest, проведена отладка в VS Code, найдены и исправлены 2 ошибки;
- выполнена интеграция модулей (catalog.py, cart.py, orders.py, analytics.py, storage.py), создан скрипт сборки build.py, написаны интеграционные тесты;
- оформлена полная документация на проект «СпортТовары»: README, API Reference, User Guide, Developer Guide, Test Report, презентация;
- создан релиз v1.0 проекта, все исходники загружены в удалённые репозитории GitHub.

Ключевые показатели:

- Все 10 работ выполнены в полном объёме.
- Все тесты пройдены успешно (17 из 17).
- Создано и задокументировано 5 учебных проектов.
- Освоено 10+ инструментов разработки.

Приобретённые навыки:

- работа с VS Code, Python 3.10+, SQLite, Git/GitHub, unittest, Markdown, Draw.io, MS Visio, MS Office;
- объектно-ориентированное и структурное проектирование;
- гибкие методы разработки (Agile, Scrum, Kanban);
- модульное и интеграционное тестирование;
- работа с системой контроля версий Git (ветвление, слияние, разрешение конфликтов, релизы);
- разработка технической документации.

Полученные знания и навыки могут быть применены при дальнейшей разработке учебных и реальных программных проектов, а также при подготовке к государственной итоговой аттестации и профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование».

Список использованных источников

1. ГОСТ 19.106-78. ЕСПД. Требования к программным документам, выполненным печатным способом.

2. ГОСТ 34.602-89. Техническое задание на создание автоматизированной системы.
3. ГОСТ 19.505-79. Руководство оператора. Требования к содержанию и оформлению.
4. ГОСТ 19.301-79. Программа и методика испытаний. Требования к содержанию и оформлению.
5. Официальная документация Python 3.10+. URL: https://docs.python.org/3/
6. Официальная документация Git. URL: https://git-scm.com/doc
7. Официальная документация Visual Studio Code. URL: https://code.visualstudio.com/docs
8. Документация модуля unittest. URL: https://docs.python.org/3/library/unittest.html
9. Документация SQLite. URL: https://www.sqlite.org/docs.html
10. Scrum Guide. URL: https://scrumguides.org/
11. Kanban Guide. URL: https://kanbanguides.org/
12. Материалы методических указаний по учебной практике УП.02 (ГБПОУ МГКЭИТ, 2026).

Приложение А. Презентация проекта «СпортТовары»

Слайд 1. Титульный

СпортТовары - система управления магазином спортивных товаров.
Автор: Тур Глеб Константинович
Группа: ЗИП-7-24
Руководитель: Наседкин М.Н.
2026 г.

Слайд 2. Цель и задачи

Цель: разработать консольное приложение на Python для управления магазином спортивных товаров.

Задачи:

- реализовать каталог товаров;
- реализовать корзину покупателя;
- реализовать оформление заказов;
- реализовать аналитику продаж;
- написать модульные тесты (unittest).

Слайд 3. Архитектура приложения

```
main.py → src/
  ├── catalog.py
  ├── cart.py
  ├── orders.py
  ├── analytics.py
  └── storage.py
```

Слайд 4. Основные функции

- **Каталог:** просмотр товаров, добавление, поиск по ID, товары с низким остатком.
- **Корзина:** добавление товара, удаление, подсчёт суммы.
- **Заказы:** создание заказа, просмотр по ID.
- **Аналитика:** общая сумма продаж, топ-5 товаров.

Слайд 5. Демонстрация работы

```
$ cd ~/SportShop1
$ python3 main.py
=== СПОРТТОВАРЫ ===
1. Каталог
2. В корзину
3. Оформить заказ
4. Аналитика
5. Выход
Выбор:
```

Слайд 6. Результаты тестирования

Модуль	**Тестов**	**Пройдено**	**Провалено**
catalog.py	5	5	0
cart.py	4	4	0
orders.py	3	3	0
analytics.py	3	3	0
storage.py	2	2	0
Итого	**17**	**17**	**0**

Слайд 7. Выводы и планы

Выводы:

- приложение «СпортТовары» полностью реализовано;
- все 17 тестов пройдены успешно;

- документация оформлена (README, API, руководства);
- создан релиз v1.0 в Git.

Планы на будущее:

- добавить GUI (Tkinter / PyQt);
- подключить веб-интерфейс (Flask);
- расширить аналитику (графики, отчёты);
- добавить экспорт в Excel / PDF.

Спасибо за внимание!

2026 г.

Слайд 2. Цель и задачи

Цель: разработать консольное приложение на Python для управления магазином спортивных товаров.

Задачи:

- реализовать каталог товаров;
- реализовать корзину покупателя;
- реализовать оформление заказов;
- реализовать аналитику продаж;
- написать модульные тесты (unittest).

Слайд 3. Архитектура приложения

```
main.py → src/
  ├── catalog.py
  ├── cart.py
  ├── orders.py
  ├── analytics.py
  └── storage.py
```

Слайд 4. Основные функции

- **Каталог:** просмотр товаров, добавление, поиск по ID, товары с низким остатком.
- **Корзина:** добавление товара, удаление, подсчёт суммы.
- **Заказы:** создание заказа, просмотр по ID.
- **Аналитика:** общая сумма продаж, топ-5 товаров.

Слайд 5. Демонстрация работы

```
$ cd ~/SportShop1
$ python3 main.py
=== СПОРТТОВАРЫ ===
1. Каталог
2. В корзину
3. Оформить заказ
4. Аналитика
5. Выход
Выбор:
```

Слайд 6. Результаты тестирования

Модуль	Тестов	Пройдено	Провалено
catalog.py	5	5	0
cart.py	4	4	0
orders.py	3	3	0
analytics.py	3	3	0
storage.py	2	2	0
Итого	**17**	**17**	**0**

Слайд 7. Выводы и планы

Выводы:

- приложение «СпортТовары» полностью реализовано;
- все 17 тестов пройдены успешно;
- документация оформлена (README, API, руководства);
- создан релиз v1.0 в Git.

Планы на будущее:

- добавить GUI (Tkinter / PyQt);
- подключить веб-интерфейс (Flask);
- расширить аналитику (графики, отчёты);
- добавить экспорт в Excel / PDF.

Спасибо за внимание!